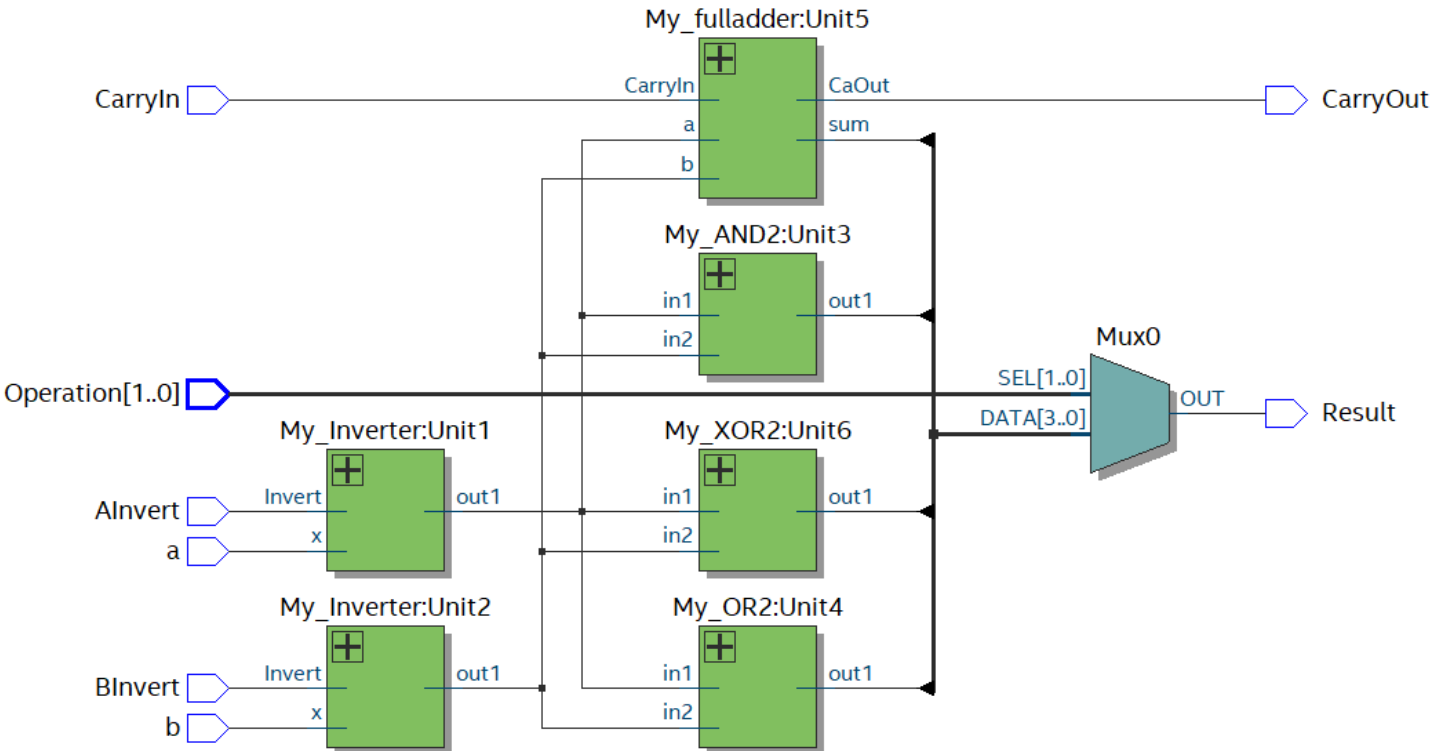
2η Ομαδική Εργασία
Σχεδίαση Ψηφιακών Συστημάτων

Μέρος 1ο - Σχεδίαση και υλοποίηση της 1-bit ALU

i. Στο φάκελο **ALU1bit** υπάρχει το project του Quartus για την 1-bit ALU σύμφωνα με τις οδηγίες της Εργασίας.

Χρησιμοποιώντας το λογισμικό Quartus, γράψαμε πηγαίο κώδικα (πρόγραμμα) στη γλώσσα VHDL, ο οποίος υλοποιεί το κύκλωμα της 1-bit ALU, με structural τρόπο χρησιμοποιώντας components, με βάση την περιγραφή των συστατικών της ALU που δίνονται στην Εργασία. Στο αρχείο της VHDL δημιουργήσαμε και το package **My_Package**, για να το χρησιμοποιήσουμε στην υλοποίηση της 16-bit ALU, στο 2ο μέρος της εργασίας.

Το RTL διάγραμμα (Tools→Netlist Viewers→RTL viewer) φαίνεται στην εικόνα 1 (υπάρχει ως screenshot στο αρχείο **ALU1bit RTL.pdf**).



Εικόνα 1: Το RTL διάγραμμα της 1-bit ALU

Όπως προκύπτει από την Εργασία, οι πράξεις της 1-bit ALU, σε σχέση με τα σήματα εισόδου φαίνονται στον πίνακα 1.

ΠΡΑΞΗ	Operation	AInvert	BInvert	CarryIn
AND	00	0	0	0
OR	01	0	0	0
ADD	10	0	0	0
SUB	10	0	1	1
NOR	00	1	1	0
NAND	01	1	1	0
XOR	11	0	0	0

Πίνακας 1: Η λειτουργία της 1-bit ALU

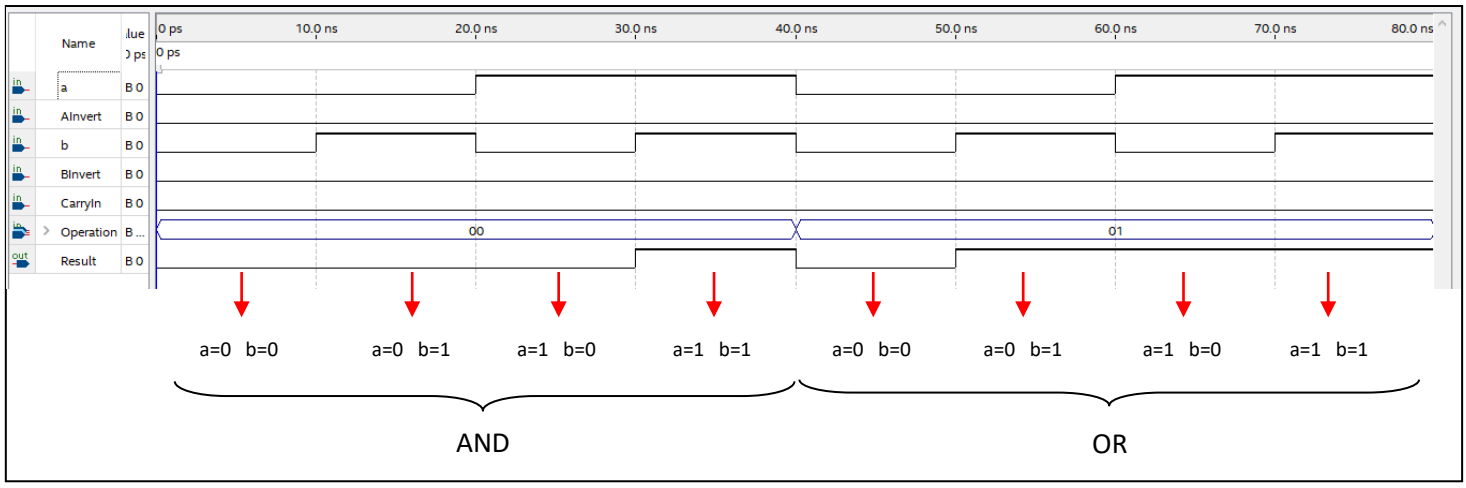
ii. Χρησιμοποιήσαμε την προσομοίωση λειτουργίας (waveforms) (File → New → Verification/Debugging Files → University Program VWF) του λογισμικού Quartus για να αποδείξουμε την ορθότητα της υλοποίησής μας για όλες τις πιθανές τιμές των a, b (σήματα του 1- bit) και τις πράξεις που υποστηρίζει η 1-bit ALU.

Για να φανούν καθαρά οι κυματομορφές τρέξαμε τη λειτουργική προσομοίωση μας (File → New → Verification/Debugging Files → University Program VWF) περισσότερες φορές: μία φορά για τις πράξεις AND και OR, άλλη για τις πράξεις ADD και SUB κι άλλη για τις πράξεις NAND, NOR και XOR.

i.Στην εικόνα 2 φαίνονται οι κυματομορφές (αρχείο **WaveformAND_OR.vwf**) για τις πράξει AND και OR, όπως φαίνεται από την εικόνα, οι κυματομορφές ικανοποιούν τον πίνακα αλήθειας 2.

a	b	AND Result	OR Result
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Πίνακας 2: Πίνακας αλήθειας AND και OR



Εικόνα 2: Οι κυματομορφές για τις πράξεις AND και OR

ii.Στην εικόνα 3 φαίνονται οι κυματομορφές (αρχείο **WaveformADD_SUB.vwf**) για τις πράξει ADD και SUB, όπως φαίνεται από την εικόνα, οι κυματομορφές ικανοποιούν τον πίνακα αλήθειας 3 για τις πράξεις που προκύπτουν από τις ακόλουθες λογικές εξισώσεις:

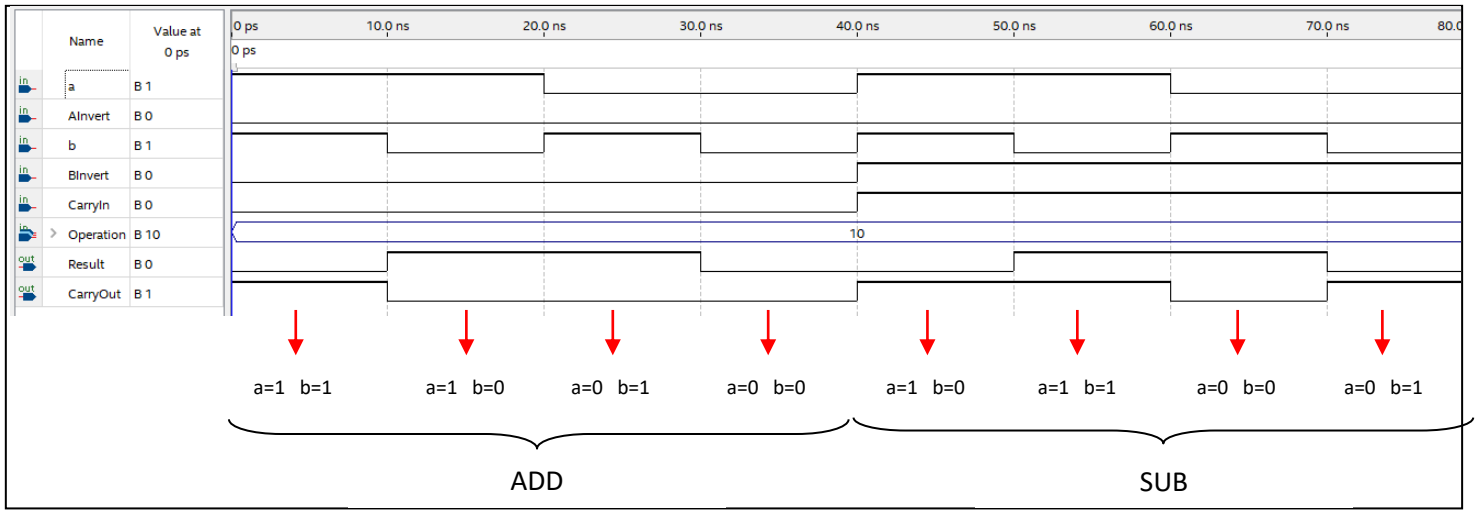
CarryOut = (b • CarryIn) + (a • CarryIn) + (a • b),

$$Sum = \left(a \bullet \bar{b} \bullet \overline{\text{CarryIn}}\right) + \left(\bar{a} \bullet b \bullet \overline{\text{CarryIn}}\right) + \left(\bar{a} \bullet \bar{b} \bullet \text{CarryIn}\right) + \left(a \bullet b \bullet \text{CarryIn}\right)$$

ADD				
a	b	CarryIn	Result (Sum)	CarryOut
1	1	0	0	0
1	0	0	1	0
0	1	0	1	0
0	0	0	0	0

SUB				
a	b	CarryIn	Result (Sum)	CarryOut
1	1	1	1	1
1	0	1	0	1
0	1	1	0	1
0	0	1	1	0

Πίνακας 3: Πίνακας αλήθειας ADD και SUB

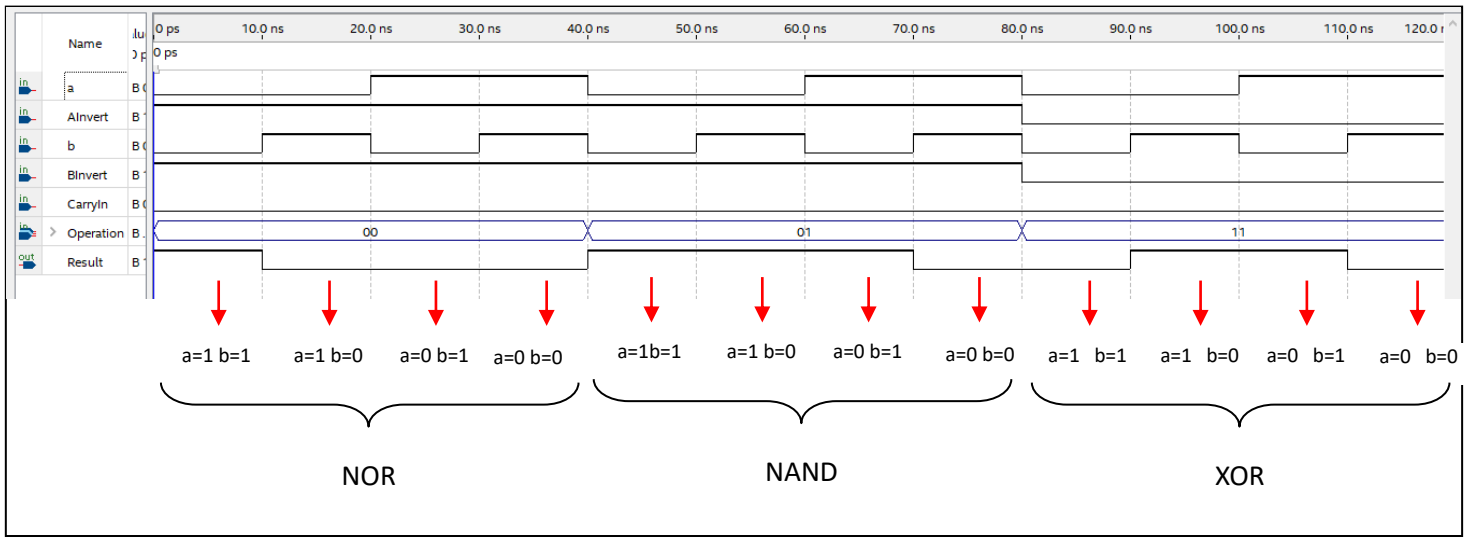


Εικόνα 3: Οι κυματομορφές για τις πράξεις ADD και SUB

iii. Στην εικόνα 4 φαίνονται οι κυματομορφές (αρχείο **WaveformNOR_NAND_XOR.vwf**) για τις πράξει NOR, NAND και XOR όπως φαίνεται από την εικόνα, οι κυματομορφές ικανοποιούν τον πίνακα αλήθειας 4.

a	b	NOR Result	NAND Result	XOR Result
1	1	0	0	0
1	0	0	1	1
0	1	0	1	1
0	0	1	1	0

Πίνακας 4: Πίνακας αλήθειας NOR, NAND και XOR



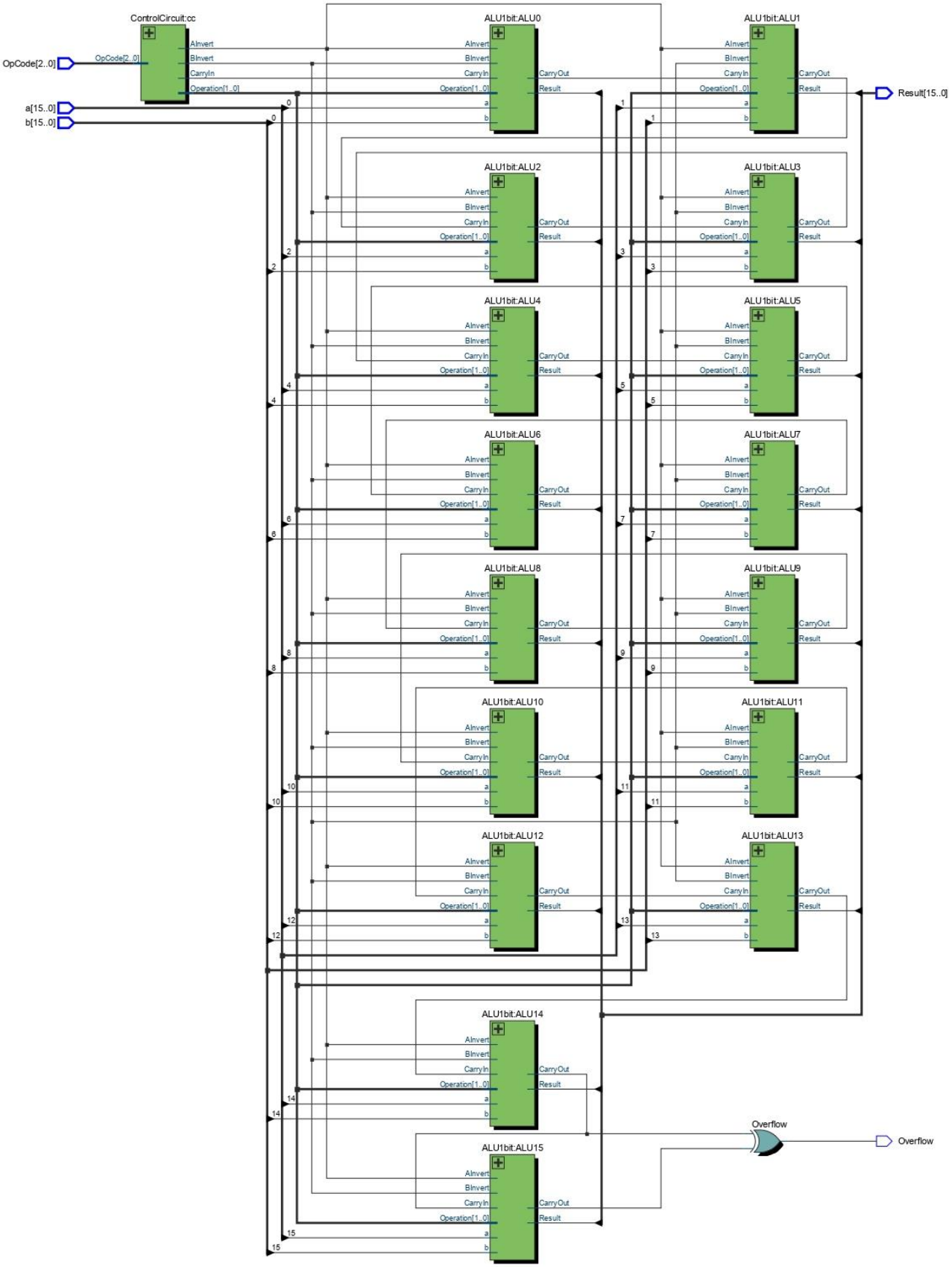
Εικόνα 4: Οι κυματομορφές για τις πράξεις NOR, NAND και XOR

Μέρος 2ο – Υλοποίηση ιεραρχικής ALU των 16-bits με slices της 1-bit ALU

i. Στο φάκελο **ALU16bit** υπάρχει το project του Quartus για την 1-bit ALU σύμφωνα με τις οδηγίες της Εργασίας. Στον φάκελο αυτό αντιγράφουμε το αρχείο ALU1bit.vhd που περιέχει το package με όνομα My_Package, το οποίο περιγράφει την 1-bit ALU ως component.

Στην συνέχεια γράψαμε πηγαίο κώδικα (πρόγραμμα) στη γλώσσαVHDL, ο οποίος υλοποιεί το κύκλωμα της 16-bit ALU, με structural τρόπο, με βάση την περιγραφή των συστατικών της ALU που δίνονται στην Εργασία.

Το RTL διάγραμμα (Tools→Netlist Viewers→RTL viewer) φαίνεται στην εικόνα 5 (υπάρχει ως screenshot στο αρχείο **ALU16bit RTL.pdf**).



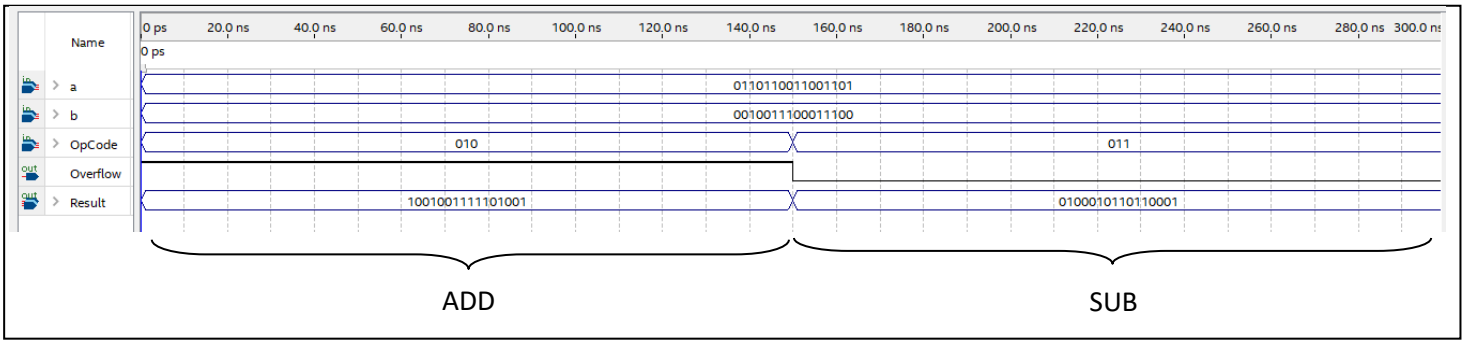
Εικόνα 5: Το RTL διάγραμμα της 16-bit ALU

ii. Προσομοίωση λειτουργίας

i. Πρόσθεση (ADD) και αφαίρεση (SUB) αριθμών (σε μορφή συμπληρώματος ως προς 2):

Περίπτωση 1: a = 0110110011001101 και b = 0010011100011100

Στην εικόνα 6 φαίνεται οι κυματομορφές (αρχείο **WaveformADD_SUB_1.vwf**) για την πρόσθεση και την αφαίρεση των δύο παραπάνω αριθμών.



Εικόνα 6: Πρόσθεση και αφαίρεση των 0110110011001101 και 0010011100011100

Για την **πρόσθεση** το αποτέλεσμα είναι **λάθος**, προκύπτει **OverFlow**

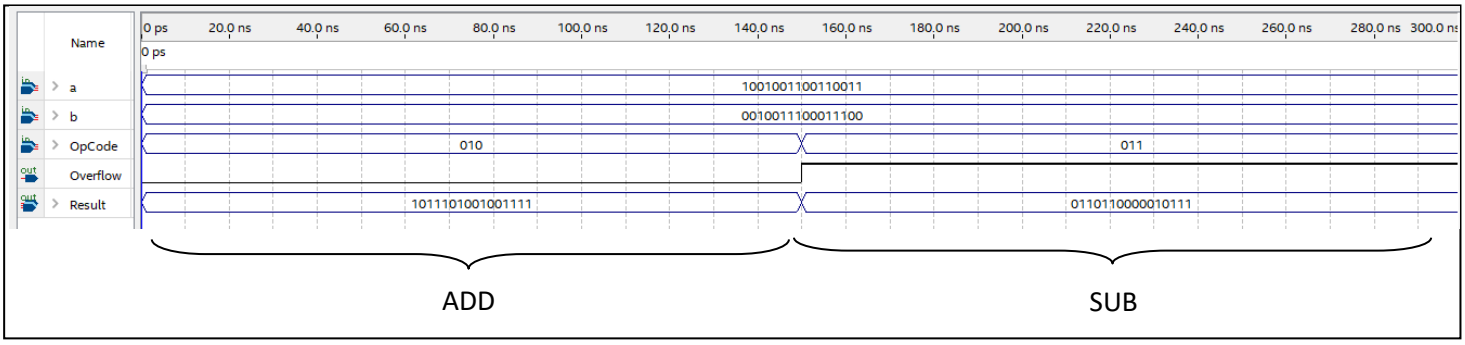
	Συμπλήρ. ως προς 2	Δεκαδική μορφή	
	0110110011001101	27853	
+	0010011100011100	10012	
	1001001111101001	-27671	αντί για 37865 (υπερχείλιση)

Για την **αφαίρεση** το αποτέλεσμα είναι σωστό:

	Συμπλήρ. ως προς 2	Δεκαδική μορφή	
	0110110011001101	27853	
-	0010011100011100	10012	
	0100010110110001	17841	σωστό αποτέλεσμα

Περίπτωση 2: a = 1001001100110011 και b = 0010011100011100

Στην εικόνα 7 φαίνεται οι κυματομορφές (αρχείο **WaveformADD_SUB_2.vwf**) για την πρόσθεση και την αφαίρεση των δύο παραπάνω αριθμών.



Εικόνα 7: Πρόσθεση και αφαίρεση των 1001001100110011 και 0010011100011100

Για την **πρόσθεση** το αποτέλεσμα είναι σωστό:

	Συμπλήρ. ως προς 2	Δεκαδική μορφή	
	1001001100110011	-27853	
+	0010011100011100	10012	
	1011101001001111	-17841	σωστό αποτέλεσμα

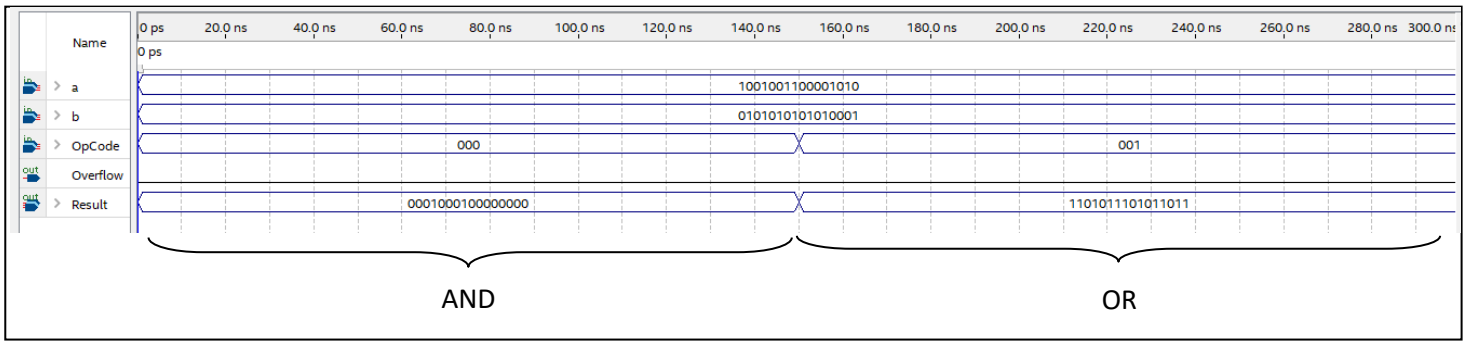
Για την **αφαίρεση** το αποτέλεσμα είναι **λάθος**, προκύπτει **OverFlow**:

	Συμπλήρ. ως προς 2	Δεκαδική μορφή
	1001001100110011	-27853

-	0010011100011100	10012	
	0110110000010111	27671	αντί για 37865 (υπερχείλιση)

ii. Λογικές πράξεις AND και OR των a, b όπου:
a = 1001001100001010
b = 0101010101010001

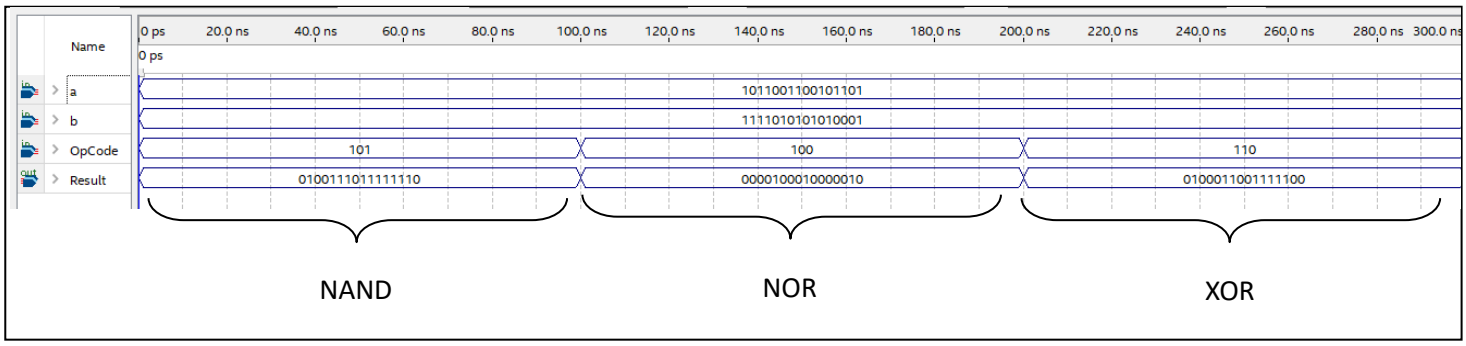
Στην εικόνα 8 φαίνονται οι κυματομορφές (αρχείο **WaveformAND_OR.vwf**) για τις πράξεις AND και OR.



Εικόνα 8: Λογικές πράξεις AND και OR των 1001001100001010 και 0101010101010001

iii. Λογικές πράξεις NAND, NOR και XOR των a, b όπου:
a = 1011001100101101
b = 1111010101010001

Στην εικόνα 9 φαίνονται οι κυματομορφές (αρχείο **WaveformNAND_NOR_XOR.vwf**) για τις πράξεις NAND, NOR και XOR.



Εικόνα 9: Λογικές πράξεις NAND, NOR και XOR των 1011001100101101 και 1111010101010001